

Trí tuệ nhân tạo

chương 2

GIẢI QUYẾT VẤN ĐỀ BẰNG TÌM KIẾM (2)

Nội dung (phần 2)

- I. Biểu diễn vấn đề trong không gian tìm kiếm
- II. Các chiến lược tìm kiếm mù
- III. Chiến lược tìm kiếm kinh nghiệm (heuristic)
- IV. Chiến lược tìm kiếm tối ưu
- V. Tìm kiếm có đối thủ

Các chiến lược tìm kiếm mù

- Trạng thái được chọn để phát triển chỉ đơn thuần dựa theo cấu trúc của KGTT mà không có thông tin hướng dẫn nào khác.
- Thông tin về TT đích không đóng vai trò trong tìm kiếm.
- Kém hiệu quả và không thể áp dụng được trong nhiều trường hợp.
- Sử dụng thông tin của trạng thái kết hợp với nhận xét dựa theo kinh nghiệm để cải tiến là quan điểm chung của các chiến lược *tìm kiếm kinh nghiệm*.

3/2018

3

Nội dung

- I. Biểu diễn vấn đề trong không gian tìm kiếm
- II. Các chiến lược tìm kiếm mù
- III. **Chiến lược tìm kiếm kinh nghiệm (heuristic)**
- IV. Chiến lược tìm kiếm tối ưu
- V. Tìm kiếm có đối thủ

3/2018

4

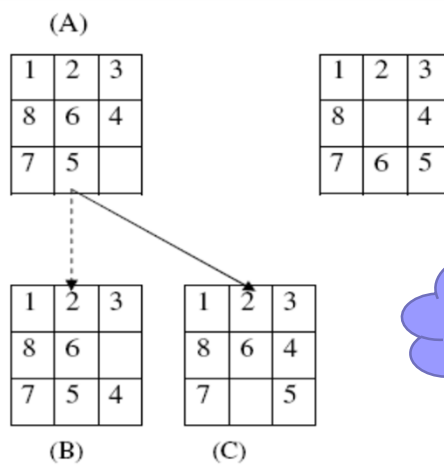
III. Chiến lược tìm kiếm kinh nghiệm (heuristic)

1. Hàm đánh giá
2. Tìm kiếm ưu tiên tốt nhất (Best First Search (Best-FS))
3. Tìm kiếm leo đồi (Hill Climbing Search –HCS)

3/2018

5

1. Hàm đánh giá



Mở rộng theo B
hay C

3/2018

6

Khái niệm về hàm đánh giá

- Hàm đánh giá là một hàm ước lượng khả năng về đích của mỗi trạng thái.
- Chỉ là ước lượng vì nói chung chúng ta không thể tính toán chính xác khả năng này, nếu tính được nghĩa là đã tìm được lời giải!
- Tuy nhiên, nhờ kinh nghiệm trong nhiều bài toán cụ thể, có thể ước lượng được giá trị này.

3/2018

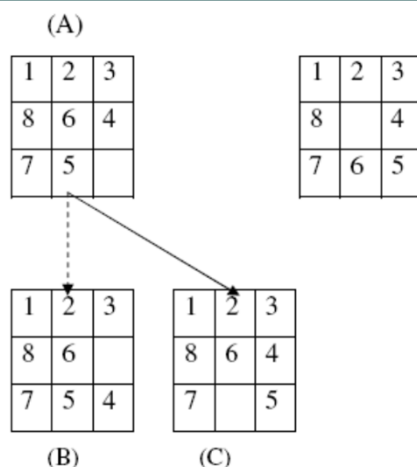
7

Ví dụ

- Nếu đếm các số sai vị trí của một trạng thái so với trạng thái đích thì B là 3 và C là 1.

- Như vậy, số này càng nhỏ thì trạng thái đó càng có khả năng về đích

- Có thể dùng số này làm giá trị cho hàm đánh giá.



3/2018

8

Hàm đánh giá

$$h: X \rightarrow R^+$$

- Trong đó X là KGTT của bài toán.
- Dễ thấy, nếu u là trạng thái đích thì $h(u)=0$. Mỗi u ta có một giá trị ước lượng là $h(u)$
ví dụ: $h(B)=3$; $h(C)=1$
- Hàm đánh giá còn được gọi là hàm heuristic. Giá trị hàm càng nhỏ thì khả năng về đích của trạng thái càng lớn.

3/2018

9

Xác định hàm đánh giá

1	2	3
8		4
7	6	5

Tính $h(D)$, $h(E)$?

Có nhiều cách
xây dựng hàm
đánh giá

1	5	3
8		4
7	6	2

(D)

1	4	3
8		2
7	6	5

(E)

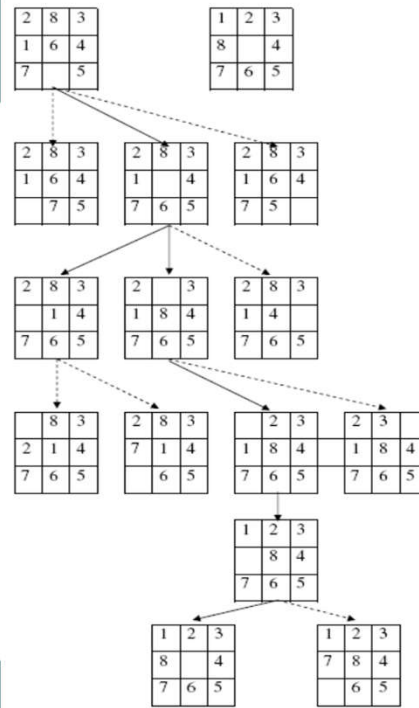
- $h(D) = h(E) = 2$?
- Về mặt đường đi qua các ô ngang dọc để di chuyển về đúng vị trí thì (D) phải di chuyển đoạn đường xa hơn (E) nên có thể là (D) không tốt bằng (E).
- Theo nhận xét này, ta có thể xây dựng hàm h bằng tổng khoảng cách phải di chuyển của các ô sai vị trí (khoảng cách mahattan).
- Ví dụ: $h_1(D)=3+3 = 6$, $h_1(E)=2+2=4$

3/2018

10

Ví dụ

Xây dựng hàm đánh giá



3/2018

Chiến lược tìm kiếm kinh nghiệm

- Các kỹ thuật tìm kiếm sử dụng hàm đánh giá để hướng dẫn sự tìm kiếm được gọi chung là các kỹ thuật tìm kiếm kinh nghiệm (heuristic search)
- Các bước giải quyết vấn đề bằng tìm kiếm kinh nghiệm
 - Biểu diễn bài toán bằng một KGTT thích hợp
 - **Xây dựng hàm đánh giá**
 - Thiết kế chiến lược chọn trạng thái

3/2018

12

Chiến lược tìm kiếm kinh nghiệm

Hai chiến lược quan trọng:

- Tìm kiếm ưu tiên tốt nhất (*best-first search*)
= Tìm kiếm theo bề rộng + Hàm đánh giá
- Tìm kiếm leo đồi (*hill-climbing search*)
= Tìm kiếm theo độ sâu + Hàm đánh giá

3/2018

13

1. Tìm kiếm ưu tiên tốt nhất Best First Search (Best-FS)

- Chọn trạng thái tốt nhất (theo hàm đánh giá) trong **open** để phát triển, kể cả trạng thái này không tốt bằng trạng thái đã sinh ra nó.
- Lưu trữ tất cả các trạng thái anh em nên khi đi vào ngõ cụt chiến lược này có thể lui ra được mà không bị bế tắc.
- Best-FS kết hợp tìm kiếm sâu và rộng.
- Best-FS “cẩn thận” hơn ghi nhớ lại một số trạng thái không tốt hơn trước đó để còn có thể quay lại.

Phương án lưu nghiệm:

- Một cách đầy đủ, để có được kết quả là đường đi nghiệm chúng ta phải lưu ý thêm về việc lưu giữ các trạng thái cha để truy lại vết của đường đi này.

3/2018

14

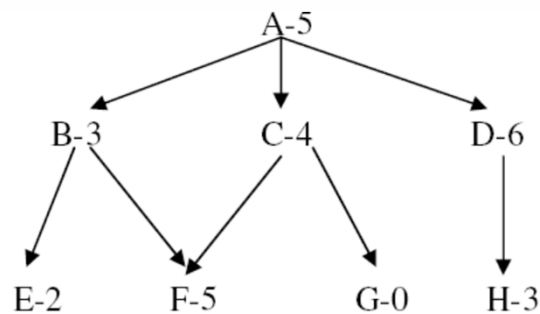
Thuật toán Best_FS

```
procedure Best_FS;  
begin  
  open:={u0}; closed:={ };  
  while open<>{ } do  
    begin  
      loại trạng thái ngoài cùng bên trái của open, gọi nó là u  
      if (u là một đích) then thông báo thành công, thoát  
      else begin  
        Đưa u vào closed  
        Phát sinh các con v của u  
        Loại các con v đã có mặt trong open + closed  
        Đưa các con còn lại vào open  
        Sắp xếp open sao cho phần tử tốt nhất nằm bên trái  
      end  
    end  
  Thông báo thất bại  
end
```

3/2018

15

- xác định các tập open và close cho cây tìm kiếm ưu tiên tốt nhất

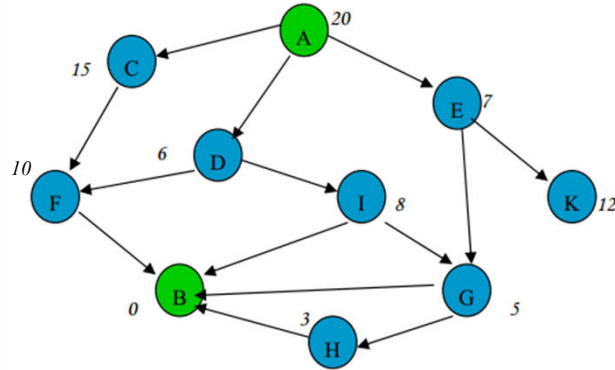


3/2018

16

Ví dụ

- xác định các tập open và closed cho cây tìm kiếm ưu tiên tốt nhất



Hình 2.2: Đồ thị không gian trạng thái

3/2018

17

3. Tìm kiếm leo đồi (Hill Climbing Search –HCS)

- Tìm kiếm leo đồi (hill-climbing search, HCS) là tìm kiếm theo độ sâu được hướng dẫn bởi hàm đánh giá.
- Khác với tìm kiếm theo độ sâu, khi phát triển một đỉnh u thì bước tiếp theo ta chọn trong số các đỉnh con của u đỉnh có nhiều hứa hẹn nhất để phát triển, đỉnh này được xác định bởi hàm đánh giá.
- Nếu không có thuật toán phải dừng.
- Nếu chỉ chọn một trạng thái *tốt hơn* -> leo đồi đơn giản, nếu chọn trạng thái *tốt nhất* -> leo đồi dốc đứng.
- Sử dụng hàm h để biết trạng thái nào tốt hơn.
- Khác với tìm kiếm sâu, leo đồi không lưu tất cả các con mà chỉ lưu đúng một trạng thái được chọn nếu có.

3/2018

18

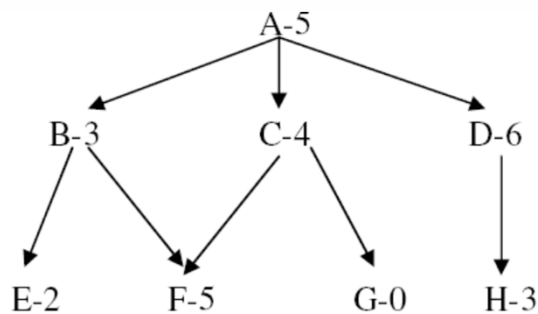
Thuật toán HCS

```
procedure hcs() ;  
begin  
  open:=[start]; closed:=[];  
  while open<>[] do  
    begin  
      loại trạng thái ngoài cùng bên trái của open, gọi nó là u  
      if (u là một đích) then thông báo kết quả, thoát  
      else begin  
        Đưa u vào closed  
        Phát sinh các con v của u  
        Loại các con đã có trong open+closed  
        Sắp xếp các con còn lại theo chiều tăng dần của hàm  
        đánh giá và đưa vào bên trái open  
      end  
    end  
  Thông báo thất bại  
End
```

3/2018

19

♦ xác định các tập open và closed cho cây tìm kiếm leo đồi

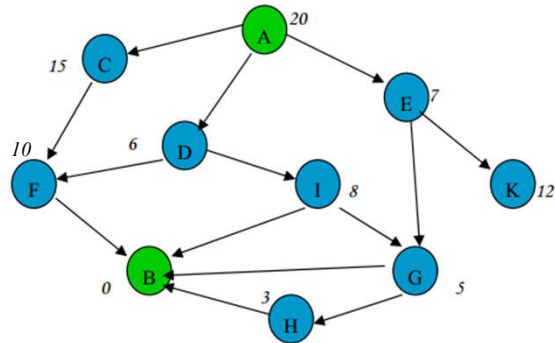


3/2018

20

Ví dụ

- ◆ xác định các tập open và closed cho cây tìm kiếm leo đồi



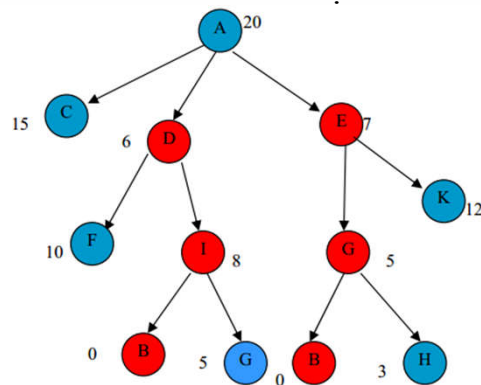
Hình 2.2: Đồ thị không gian trạng thái

3/2018

21

Tìm kiếm BEAM

- Giống như tìm kiếm theo bề rộng, tuy nhiên tại mỗi mức ta hạn chế chỉ phát triển k đỉnh tốt nhất dựa vào hàm đánh giá
- Ví dụ: $k=2$



Hình 2.5: Cây tìm kiếm Beam

3/2018

22

Nội dung

- I. Biểu diễn vấn đề trong không gian tìm kiếm
- II. Các chiến lược tìm kiếm mù
- III. Chiến lược tìm kiếm kinh nghiệm (heuristic)
- IV. **Chiến lược tìm kiếm tối ưu**
- V. Tìm kiếm có đối thủ

3/2018

23

Chiến lược tìm kiếm tối ưu

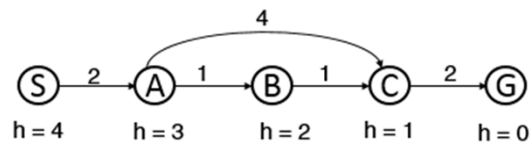
- 1. Giải thuật A*
- 2. Thuật toán tìm kiếm nhánh-và-cận

3/2018

24

Lý do

- Trong các chiến lược tìm kiếm trên, chúng ta chưa quan tâm đến độ dài hay chi phí của đường đi nghiệm.
- Trong thực tế, đây là một yếu tố quan trọng vì ý nghĩa của nghiệm tìm được phụ thuộc vào yếu tố này.
- Ví dụ: như số bước đi của một người chơi cờ; Chi phí một hành trình trong bài toán người du lịch.



- Vì vậy chúng ta không chỉ quan tâm đến việc tìm ra nghiệm mà còn phải quan tâm đến việc nghiệm đó có tối ưu hay không.

3/2018

25

Tổng quát:

- Mỗi đối tượng x trong không gian tìm kiếm được gắn với một số đo giá trị của đối tượng đó $f(x)$, mục tiêu của ta là tìm đối tượng có giá trị $f(x)$ lớn nhất (hoặc nhỏ nhất) trong không gian tìm kiếm. Hàm $f(x)$ được gọi là hàm mục tiêu.
- KGTT trong bài toán tối ưu phải có thêm trọng số của các cung.
- Chúng ta đã phân tích các chiến lược từ tìm kiếm mù, tìm kiếm kinh nghiệm và biết được những ưu nhược điểm của từng chiến lược.
- Best-FS là chiến lược tìm kiếm linh hoạt nhất cho đến lúc này, cải tiến chiến lược này để giải quyết vấn đề tối ưu.

3/2018

26

Hàm đánh giá cải tiến

- Trong Best-FS, u được ưu tiên phát triển nếu như nó có giá trị $h(u)$ nhỏ nhất trong các trạng thái trong **open**.
- Tuy nhiên, chỉ riêng $h(u)$ là không đủ khi xét đường đi tối ưu.
- Với quan điểm tối ưu, đồng thời với $h(u)$ nhỏ để mau về đích thì độ dài đường đi từ trạng thái xuất phát đến u cũng phải nhỏ. Vì vậy, người ta dùng hàm đánh giá cải tiến sau:

$$f(u) = g(u) + h(u)$$

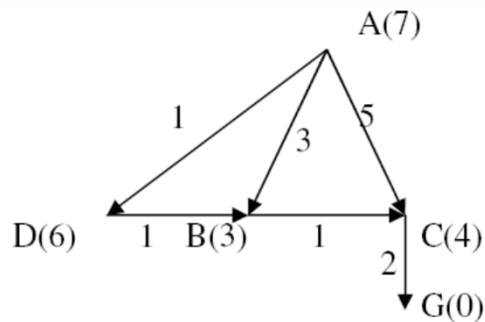
- Trong đó:
 - $g(u)$ là độ dài đường đi ngắn nhất từ u_0 đến u
 - $h(u)$ là đánh giá độ tốt theo hàm heuristic.
- Hàm $h(u)$ được gọi là chấp nhận được (hay hàm đánh giá thấp) nếu như $h(u)$ nhỏ hơn độ dài đường đi ngắn nhất thực sự từ u đến đích.

3/2018

27

Ví dụ

- Trạng thái ban đầu là trạng thái A, trạng thái đích là G, các số ghi cạnh các cung là độ dài cung đó, các số cạnh các đỉnh là giá trị của hàm h .



3/2018

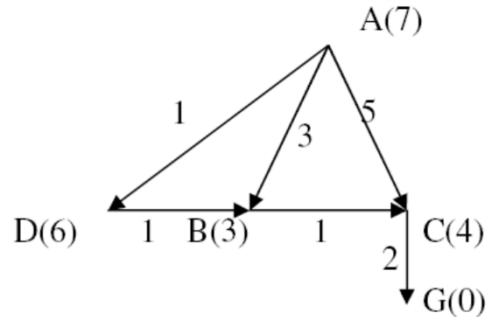
28

Thông tin mỗi nút

- Mỗi trạng thái u tùy ý sẽ gồm 4 yếu tố ($g(u)$, $h(u)$, $f(u)$, $cha(u)$). Trong đó:
 - $g(u)$, $h(u)$, $f(u)$ đã biết
 - $Cha(u)$ là nút cha của nút u

- Ví dụ

$A(0,7,7,-)$,
 $B(3,3,6,A)$,



3/2018

29

1. Giải thuật A*

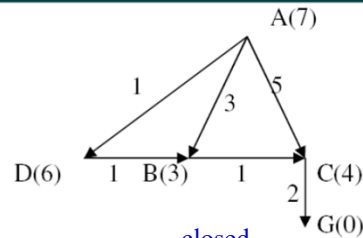
- A* là giải thuật tổng quát hơn Best-FS, nó tìm kiếm trên KGTT là đồ thị.
- Vì là đồ thị nên phát sinh nhiều vấn đề khi tìm đường đi tối ưu.
- Để ý rằng nghiệm là đường đi nên ta phải lưu lại vết của đường đi này.
- Trong các giải thuật trước, để tập trung cho tư tưởng chính của các giải thuật đó chúng ta bỏ qua chi tiết này, nhưng trong giải thuật này chi tiết này được đề cập vì nó liên quan đến nghiệm một cách trực tiếp.

3/2018

30

Mô tả hoạt động của A*

Lưu các trạng thái



Bước	Open	closed
1	A(0,7,7,-)	-
2	B(3,3,6,A), D(1,6,7,A), C(5,4,9,A)	A(0,7,7,-)
3	D(1,6,7,A), C(4,4,8,B)	B(3,3,6,A)
4	B(2,3,5,D), C(4,4,8,B)	D(1,6,7,A)
5	C(3, 4,7,B)	B(2,3,5,D)
6	G(5,0,5,C)	C(3, 4,7,B)

G(5,0,5,C) : ĐÍCH => DỪNG

Giải thuật dừng ở bước 7 và đường đi thu được độ dài 5 như sau A-D-B-C-G.

3/2018

31

Chi tiết các bước

- Ở bước 2, mọi việc xảy ra bình thường
- Ở bước 3, tìm được đường đi đến C qua B ngắn hơn nên các giá trị của C trong open phải được sửa đổi.
- Ở bước 4, mặc dù B đã nằm trong closed, tức đã xét xong nhưng đường đi mới qua D đến B ngắn hơn nên B phải được lấy khỏi closed chuyển qua open chờ xét lại với giá trị mới.
- Ở bước 5, lại xảy ra việc chỉnh sửa các giá trị của C như ở bước 3.
- Giải thuật dừng ở bước 7 và đường đi thu được độ dài 5 là: A-D-B-C-G.

Lưu ý:

- ◆ Trạng thái đã lưu trong **open** vẫn có thể thay đổi giá trị.
- ◆ Thậm chí lưu trong **closed** cũng phải bị xóa đi và chuyển qua **open** với các giá trị mới.

3/2018

32

Giải thuật A*

```
procedure A*(uo) ;  
begin  
  g(uo)=0; f(uo)=h(uo) ;  
  open:=[uo]; closed:=[] ;  
  while open<>[] do  
    begin  
      loại u ngoài cùng bên trái của open  
      Đưa u vào closed;  
      if (u là một đích) then thông báo thành công, thoát else
```

3/2018

33

```
begin  
  Sinh các con v của u;  
  For v thuộc con(u) do begin  
    g(v):=g(u)+c[u,v] ;  
    If v không thuộc open hay closed  
      begin  
        f(v):=g(v)+h(v) ;  
        cha(v):=u ;  
        Bỏ v vào open ;  
      end  
    If v thuộc open (tồn tại v' thuộc open, sao cho v=v' )  
      If g(v)<g(v') then  
        Begin  
          g(v'):=g(v) ;  
          f(v'):=g(v')+h(v') ;  
          Cha(v'):=u ;  
        End;  
  end
```

3/2018

34

```

If  $v$  thuộc closed (tồn tại  $v'$  thuộc closed, sao cho  $v=v'$ )
  If  $g(v) < g(v')$  then
    Begin
       $f(v) := g(v) + h(v)$  ;
       $cha(v) := u$  ;
      Đưa  $v$  vào open;
      Loại  $v'$  khỏi closed;
    End;
  End; {for}
  Sắp xếp open để t.thái tốt nhất nằm bên trái;
  End; {else}
  End; {while}
  Thông báo thất bại
End; {procedure}

```

3/2018

35

Kết quả

- Một hàm đánh giá $h(u)$ được gọi là chấp nhận được (hay là hàm đánh giá thấp) nếu như $h(u) \leq h^*(u)$ với mọi u , ở đây $h^*(u)$ là đường đi ngắn nhất từ u đến đích.
- Nếu hàm đánh giá $h(u)$ là chấp nhận được thì thuật toán A^* là tối ưu.
- A^* là thuật toán hiệu quả nhất trong các thuật toán đầy đủ và tối ưu.

3/2018

36

2. Thuật toán tìm kiếm nhánh-và-cận

- Branch-and-bound: Sử dụng tìm kiếm leo đồi với hàm đánh giá $f(u)$

3/2018

37

Hoạt động:

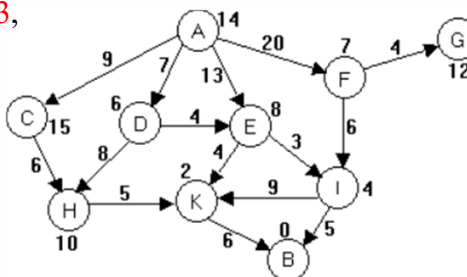
- Tại mỗi bước khi phát triển trạng thái u , thì ta sẽ chọn trạng thái tốt nhất v ($f(v)$ nhỏ nhất) trong số các trạng thái kề u để phát triển ở bước sau.
- Đi xuống cho tới khi gặp trạng thái v là đích, hoặc gặp trạng thái v không có đỉnh kề, hoặc gặp trạng thái v mà $f(v)$ lớn hơn độ dài đường đi tối ưu tạm thời.
- Trong các trường hợp này, ta không phát triển đỉnh v nữa, hay nói cách khác, ta cắt đi các nhánh cây xuất phát từ v , và quay lên cha của v để tiếp tục đi xuống trạng thái tốt nhất trong các trạng thái còn lại chưa được phát triển.

3/2018

38

VD: cây tìm kiếm nhánh và cận

- Phát triển đỉnh A, ta nhận được các đỉnh con $f(C) = 24$, $f(D) = 13$, $f(E) = 21$, $f(F) = 27$
- phát triển D, sinh ra các đỉnh con $f(H) = 25$, $f(E) = 19$
- phát triển E, sinh ra các đỉnh con $f(K) = 17$, $f(I) = 18$
- phát triển K sinh ra đỉnh B với $f(B) = g(B) = 21$.
- vì B là đỉnh đích, vậy ta tìm được đường đi tối ưu tạm thời với độ dài 21

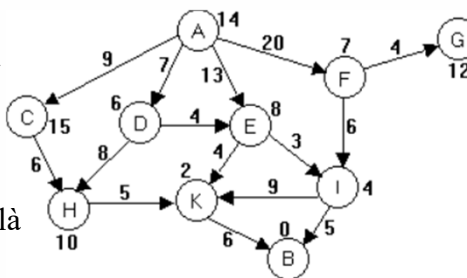


3/2018

39

VD: cây tìm kiếm nhánh và cận

- Từ B quay lên K, rồi quay lên E.
- Từ E đi xuống I, $f(I) = 18$ nhỏ hơn độ dài đường đi tạm thời (là 21). Phát triển I sinh ra các con $f(K) = 25$, $f(B) = g(B) = 19$.
- vì đỉnh B là đích ta tìm được đường đi đầy đủ mới với độ dài là 19 nhỏ hơn độ dài đường đi tối ưu tạm thời cũ (21).
- Vậy độ dài đường đi tối ưu tạm thời bây giờ là 19

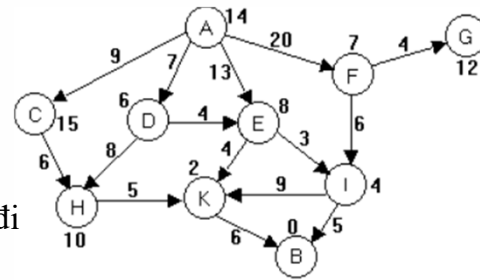


3/2018

40

VD: cây tìm kiếm nhánh và cận

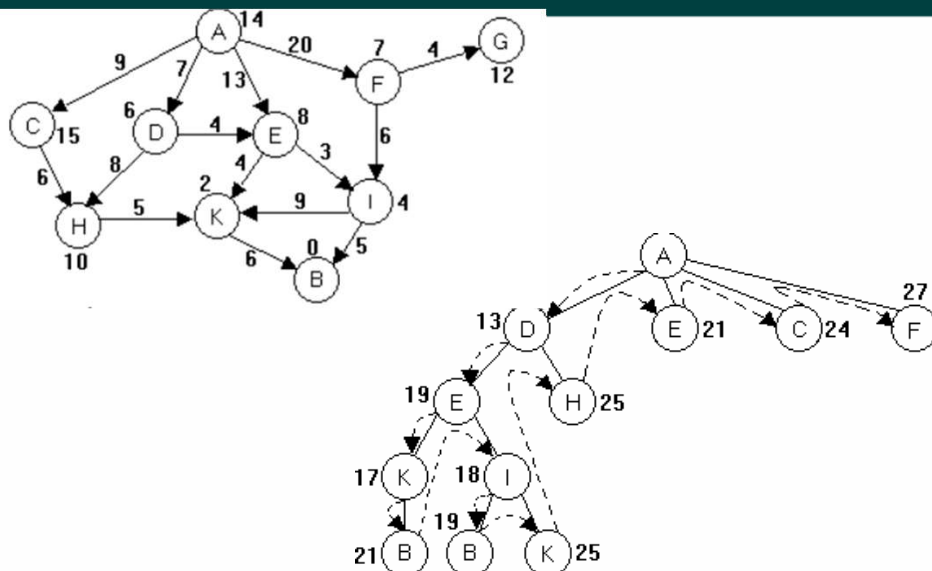
- từ B ta lại quay lên các đỉnh còn lại chưa được phát triển.
- Song các đỉnh này đều có giá trị hàm đánh giá lớn hơn 19,
- Như vậy, ta tìm được đường đi tối ưu với độ dài 19



3/2018

41

VD: cây tìm kiếm nhánh và cận



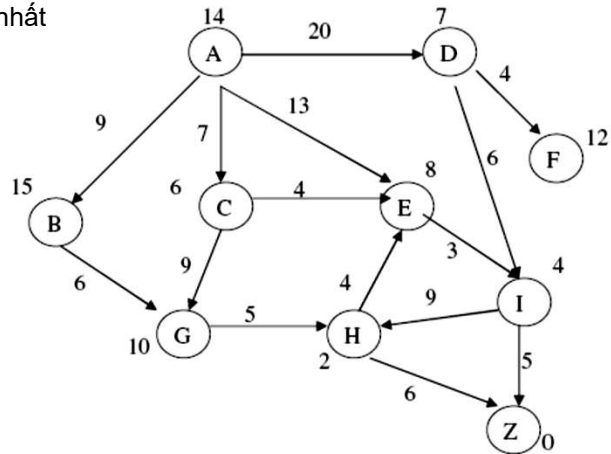
3/2018

42

Bài tập

♦ Tìm đường đi ngắn nhất từ A tới Z theo:

1. Tìm kiếm theo chiều rộng
2. Tìm kiếm theo chiều sâu
3. Tìm kiếm ưu tiên tốt nhất
4. Tìm kiếm leo đồi
5. A*
6. Nhánh – cận



3/2018